

RAJALAKSHMI ENGINEERING COLLEGE

CS23333
Object Oriented Programming Using JAVA

Contents

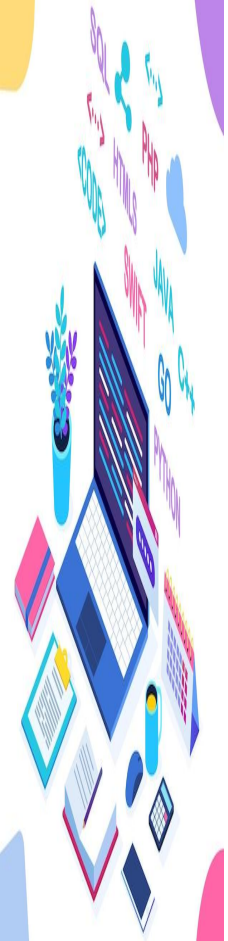
UNIT-II CLASSES AND INHERITANCE

01 Packages

02 Access Protection

03 Interfaces

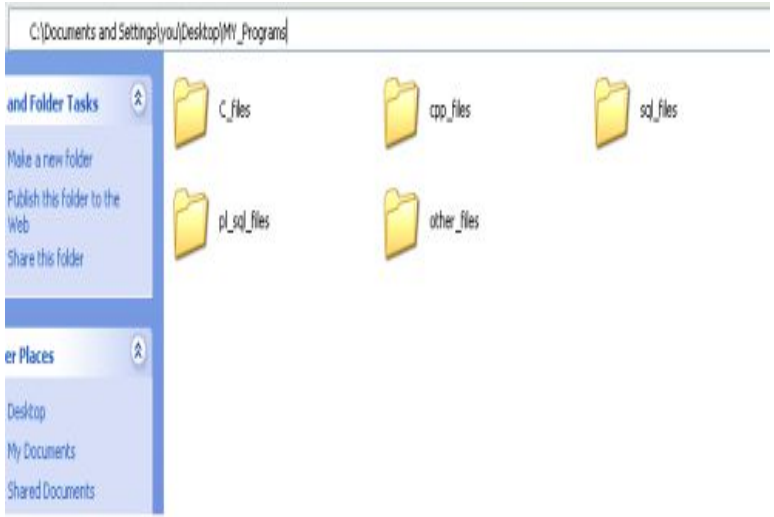
04 Exception Handling

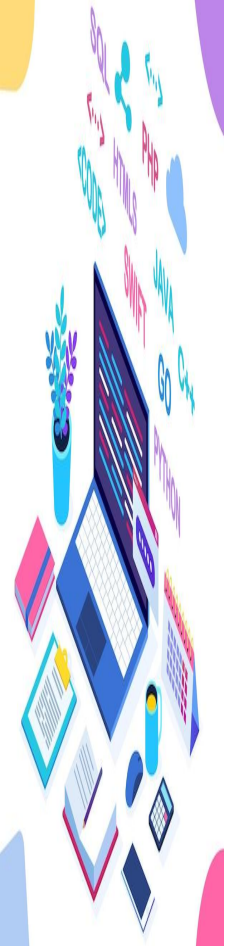


Package is similar to folders in your Disk

People normally group their related data in various folders.

- For example, a programmer may group his programs into the following folders.
  - C_programs, CPP_programs, SQL_queries, PLSQL_programs.. etc.
- We also use sub-folders for organizing our data more conveniently.
  - The advantage is we can easily locate the files if they are organized.
- Just relate package concept with directories concept in your file system

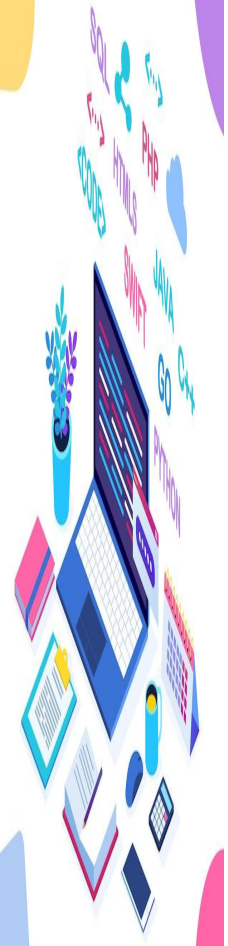




Package is similar to folders in your Disk

Need for packages

- Till now, you did not use any package since all your classes were stored in the default package. Imagine a situation where all the classes are stored in one package. It would lead to tremendous confusion as it may lead to classes with similar names that is not allowed by the language.
- This scenario is also referred to as a namespace collision.



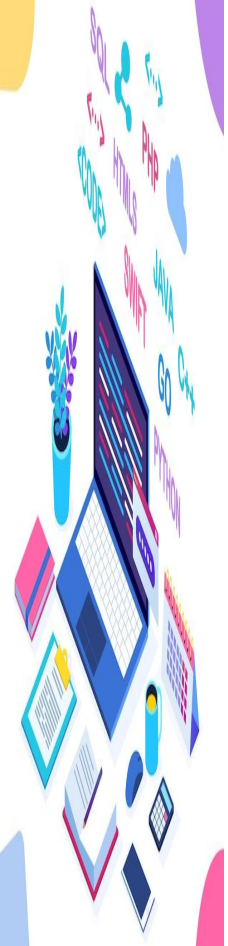
Packages

Organizing classes into Packages

- Packages are containers for classes and interfaces
- Classes and interfaces are grouped together in containers called packages
- To avoid namespace collision, we put the classes into separate containers called packages
- Whenever you need to access a class, you access it through its package by prefixing the class with the package name
- Packages are containers used to store the classes and interfaces into manageable units of code.
- Packages also help control the accessibility of your classes. This is also called as visibility control.

Example:

```
package MyPackage;
class MyClass {...}
class YourClass{...}
```



Access Protection using Packages

Packages facilitate access-control

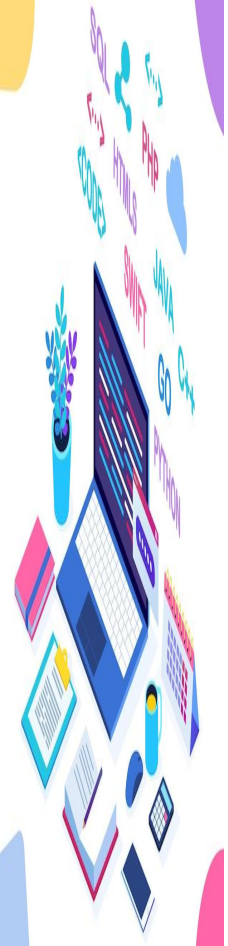
- Once a class is packaged, its accessibility is controlled by its package
- That is, whether other classes can access the class in the package depends on the access specifiers used in its class declaration
- There are four visibility control mechanisms packages offer:
  - private
  - no-specifier (default access)
  - protected
  - public



Access Protection using Packages

Packages and access-control

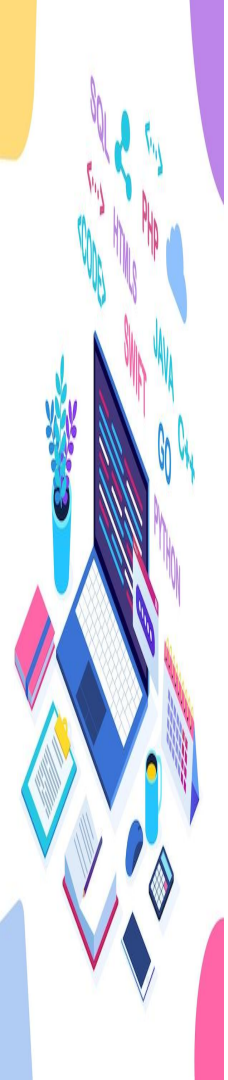
Specifier	Accessibility	
public	Subclasses and non-subclasses in the same package, as well as subclasses and non-subclasses in other packages. In other words, total visibility	Anywhere in the program
private	Accessible in the same class only	same class only
No-specifier (default access)	Subclasses and non-subclasses in the same package	same package only
protected	Subclasses and non-subclasses in the same package, and subclasses in other packages	same package and subclasses



Access Protection using Packages

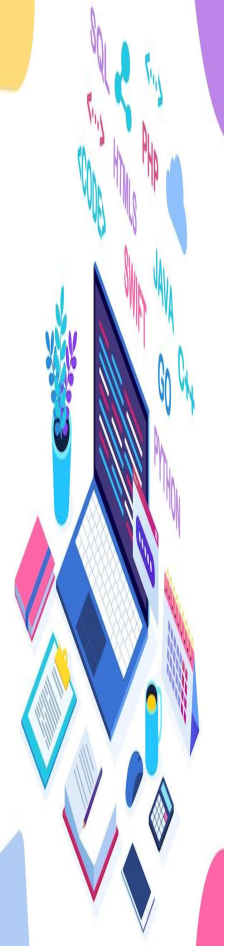
Real-time Example

- In Mini's facebook Profile page, Mini would do not wish to share some personal information like "age". But Mini would like to share "email id" with friends and Mini would like everyone to access "school/university" details.
- **How can it be solved?**
  - **Private Access:** age
  - **Protected Access:** email id
  - **Public Access:** school/university



Access Specifiers

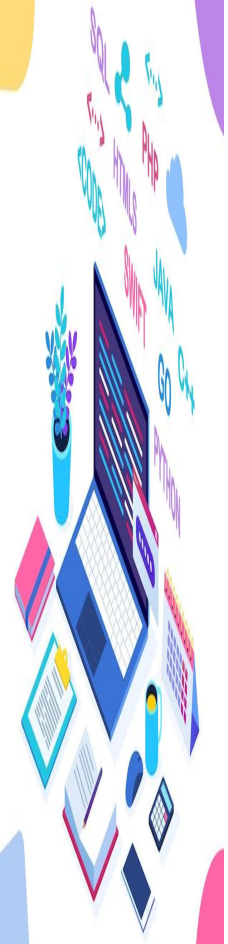
Access Specifier	Private	Default	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	No	Yes	Yes	Yes
Same Package Non-subclass	No	Yes	Yes	Yes
Different Package subclass	No	No	Yes	No
Different Package Non-Subclass	No	No	No	Yes



Inbuilt Packages

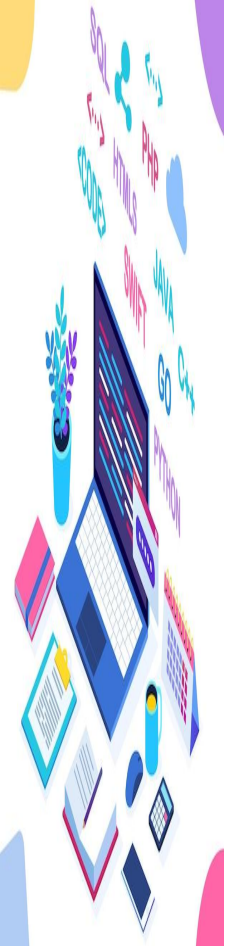
java language has various in-built packages.

- java.lang, java.io, java.util, java.awt, java.applet, java.sql, javax.swing are some of the in-built packages.
- java.lang – Basic package which is automatically imported in all programs.
  - PrintWriter, String, StringBuilder, StringBuffer, All Wrapper Classes (totally 8),
  - Thread, Runnable,
  - Throwable , Exception etc
- java.io –Input / Output related classes are available here.
  - Scanner
  - File, FileReader, FileWriter
  - BufferedReader
  - InputStreamReader
  - IOException, FileNotFoundException etc



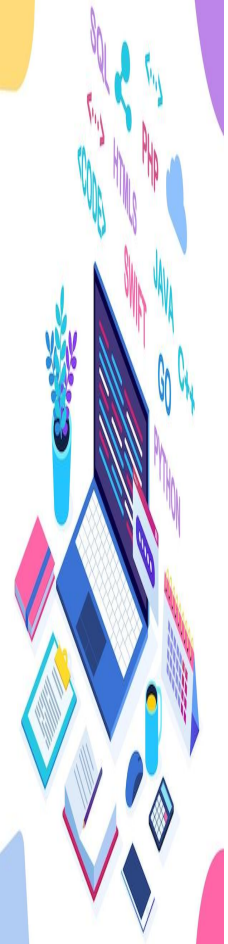
Inbuilt Packages

- java.util – Utility classes are available here. We can use these ready-made classes.
  - Date (to work with Date) , Calendar (improved one)
  - Stack (LIFO) , Queue (FIFO) , Vector , ArrayList
  - Set, HashMap etc
- java.awt – Abstract Windowing Toolkit package.
  - Various classes like Button, Label, TextArea, Menu, MenuItem are available here.
- javax.swing – Swing package.
  - Various classes like JButton, JLabel, JTextArea, JMenu, JMenuItem, JTable are available here.
- java.sql – classes used for JDBC programming
  - Various classes like Connection, DriverManager, ResultSet, SQLException are available here.



Packages & import statement

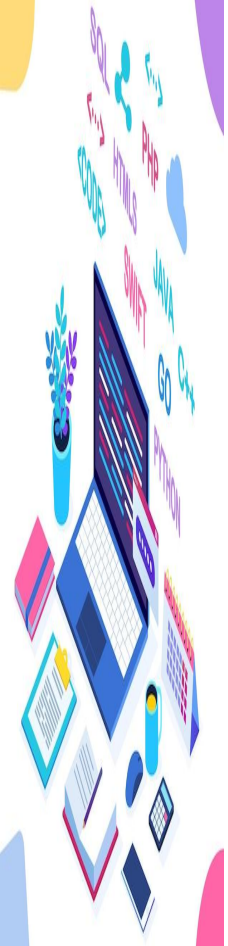
- Naturally, after creating the packages, we need to use them in our programs. Java provides import statement.
  - Import means, we can including the classes and interfaces of existing packages into our programs.
- For example,
  - import java.awt.*; -- this will be importing awt package
  - import java.awt.event.*; -- this will be importing event package which is a sub package under awt package.
- If you need a sub package, then, you need to issue a separate import statement.



Packages & import statement

Understanding CLASSPATH

- CLASSPATH is an environment variable that tells the Java runtime system where the classes are present
- When a packages is not created, all classes are stored in the default package
- The default package is stored in the current directory.
- The current directory is the default directory for CLASSPATH.
- When you create your own package for example MyPack, all the .class files including MyClass are saved in the directory MyPack.
- In order for a program to find MyPack, one of two things must be true:
  - Either the program is executed from a directory immediately above MyPack, or
  - CLASSPATH must be set to include the path to MyPack



Creating our own Package Example

```
package empPack;
class EmpClass{
String empName;
double salary;
EmpClass(String name, double sal){
empName = name;
salary = sal;
}
void display(){
System.out.println(empName + " : $"+salary);
}
}
```

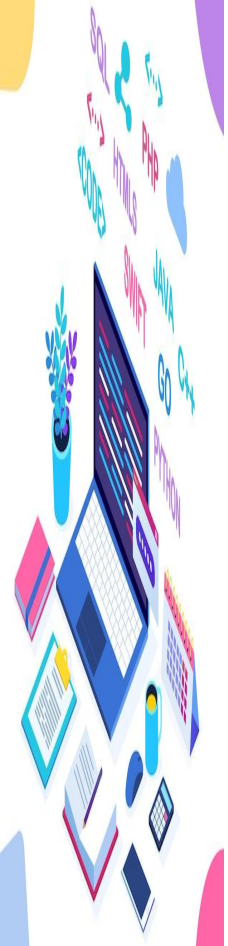
```
class EmpSal{
public static void main(String args[]){
EmpClass emp[] = new EmpClass[4];
emp[0] = new EmpClass("Bill Gates",450.20);
emp[1] = new EmpClass("D.M Ritchie", 725.93);
emp[2] = new EmpClass("Tagore",630.80);
emp[3] = new EmpClass("Kalam",545.60);
for (int i=0; i<4; i++)
emp[i].display();
}
}
```

How you will save this file?

In command prompt:

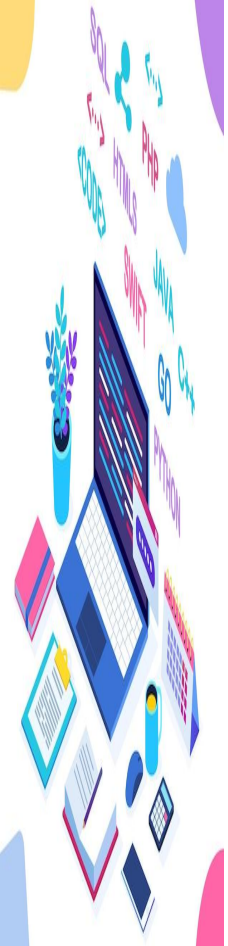
How you will compile?

How you will run?



Importing Classes from Packages

- Java has used the package mechanism extensively to organize classes with similar functionality in one package
- If you want to use these classes in your applications, you can do so by including the following statement at the beginning of your program:
  - import packagename.classname;
- If the packages are nested you should specify the hierarchy.
  - import package1.package2.classname;
- The class you want to use must be qualified by its package name.
- If you want to use several classes from a package, it would be cumbersome to type so many classes qualified by their packages.
- It can be made easy by giving a star(*) at the end of the import statement. For example:
  - import package1.*;



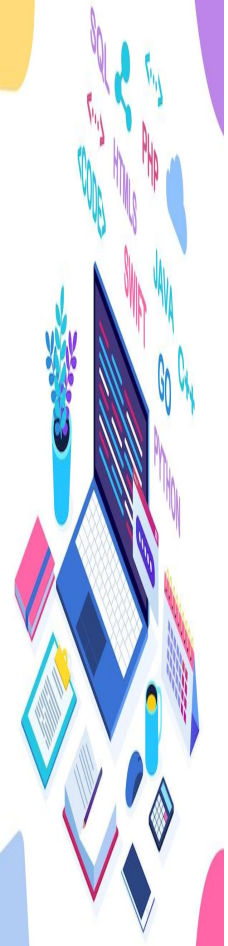
Static Import

- A static import declaration enables us to refer to imported static members as though they were declared in the current class
- If we use static import, we first have to import this static member in the following way :

```
package p1;
public class Abc {
public static void xyz() {
System.out.println("static import demo");
}
}
```

```
package p2;
import static p1.Abc.xyz;
public class A1 {
public static void main(String[] args) {
xyz();
}
}
```

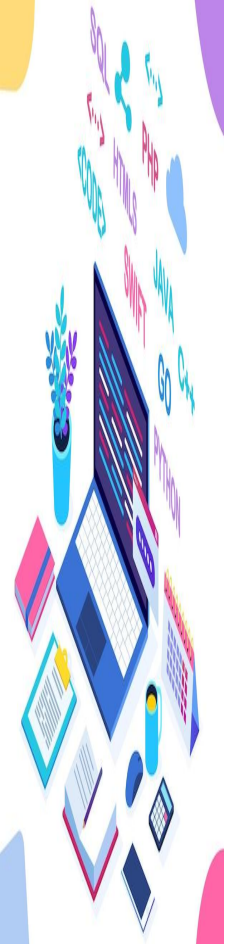
Output : "static import demo"



Static Import

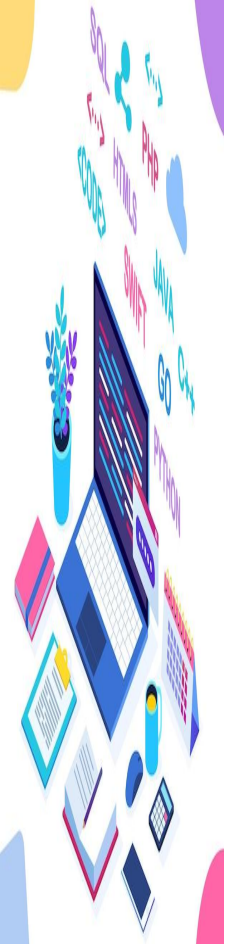
- If we are invoking multiple static members of the same class, we can also use asterisk(*), which indicates that all static members of the specified class should be available for use

```
import static java.lang.Math.*;
public class StaticImportDemo {
static float x = 4.556f;
static double y =4.556;
public static void main( String args[] ) {
float a1 = abs(x);
int r1 = round(x);
double s1 = sqrt(y);
System.out.println("absolute value of "+x+" is" +a1);
System.out.println("When we round off "+x+"we get" +r1);
System.out.println("Square Root of "+y+ "is" +s1);
}
}
```



Creating our own Packages

- We can create our own packages in java
  - Package statement helps us to create our own package.
  - Package statement should be the first statement in your program.
  - We group related classes and interfaces into a package
  - We can have sub-packages inside our packages as required
- Packages are stored as directories in Hard disk:
  - Remember, the case should match exactly
  - Look at the program in next page & try it from command line:

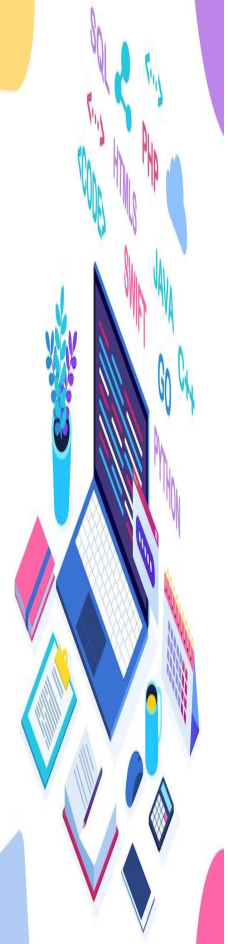


Working with Packages

```
package automobile;

public class Vehicle {
public void printname() {
        System.out.println("My name is vehicle");
        System.out.println(" I am defined inside automobile package");
}
}
```

What is the package name?
How you will save this file?

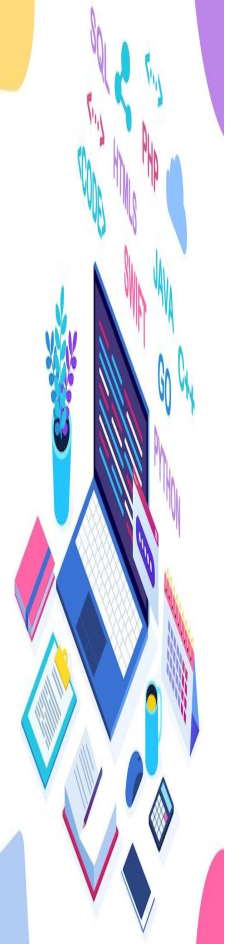


Working with Packages

```
package automobile;

public class Bike extends Vehicle {
public void printname() {
        System.out.println("My name is bike");
        System.out.println(" I am defined inside automobile package");
}
}
```

What is the package name?
How you will save this file?



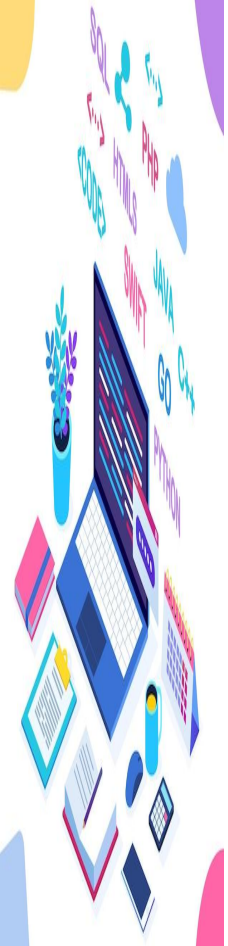
Working with Packages

```
package automobile;

public class Car extends Vehicle {
public void printname() {
        System.out.println("My name is car");
        System.out.println(" I am defined inside automobile package");
}
}
```

What is the package name?
How you will save this file?

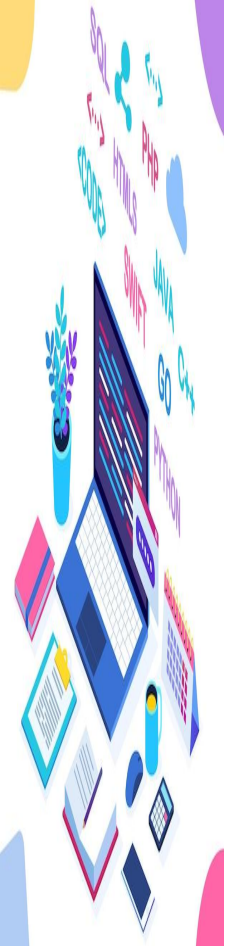
Working with Packages



```
package au_test;
import automobile.*;
public class tester {
public static void main(String s[ ] ) {
System.out.println(" I am tester class defined inside au_tester package");
System.out.println(" I had imported all classes of automobile package");
System.out.println(" Creating instances of Vehicle, Car and Bike ");
Vehicle v = new Vehicle();
Car c = new Car();
Bike b = new Bike();
System.out.println(" Accessing the functions using objects");
v.printname();
c.printname();
b.printname();
}
}
```

How you will save this file?
In command prompt:
How you will compile?
And How you will run?

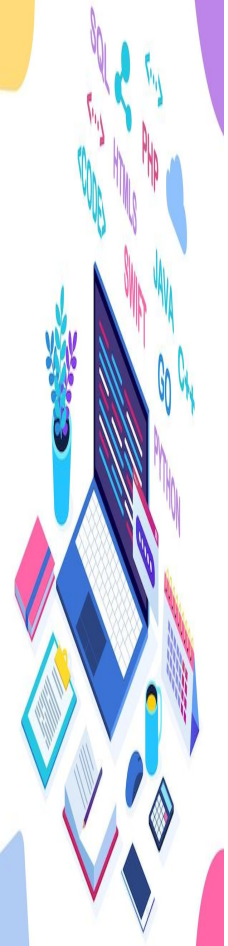
What is the output of the program?



What will be the result, when you try to compile and execute :

```
class A1 {
protected void m1() {
        System.out.println("m1 method of class A1");
}
}
class A2 extends A1 {
void m1() {
        System.out.println("m1 method of class A2");
}
public static void main(String[] args) {
        A2 x = new A2();
        x.m1();
}
}
```

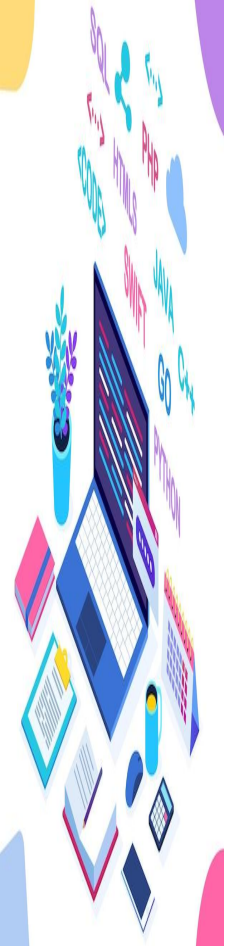
Compilation Error...Why?



What will be the result, when you try to compile and execute :

```
class A1 {
protected void m1() {
        System.out.println("m1 method of class A1");
}
}
class A2 extends A1 {
public void m1() {
        System.out.println("m1 method of class A2");
}
public static void main(String[] args) {
        A2 x = new A2();
        x.m1();
}
}
```

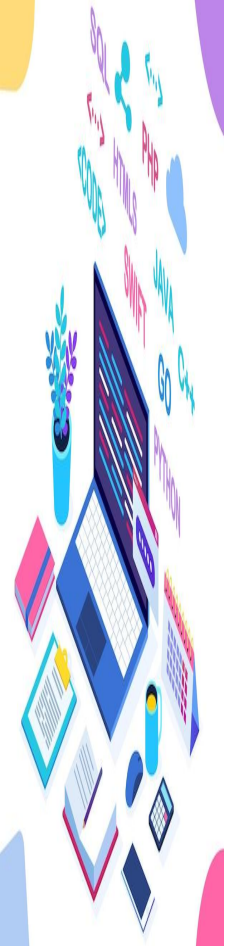
The code compiles and executes successfully! Prints "m1 method of class A2"



What will be the result, when you try to compile and execute :

```
class A1 {
protected void m1() {
        System.out.println("m1 method of class A1");
}
}
class A2 extends A1 {
void m1(int i) {
        System.out.println("m1 method of class A2");
}
public static void main(String[] args) {
        A2 x = new A2();
        x.m1();
}
}
```

The code compiles and executes successfully..! Prints "m1 method of class A1"

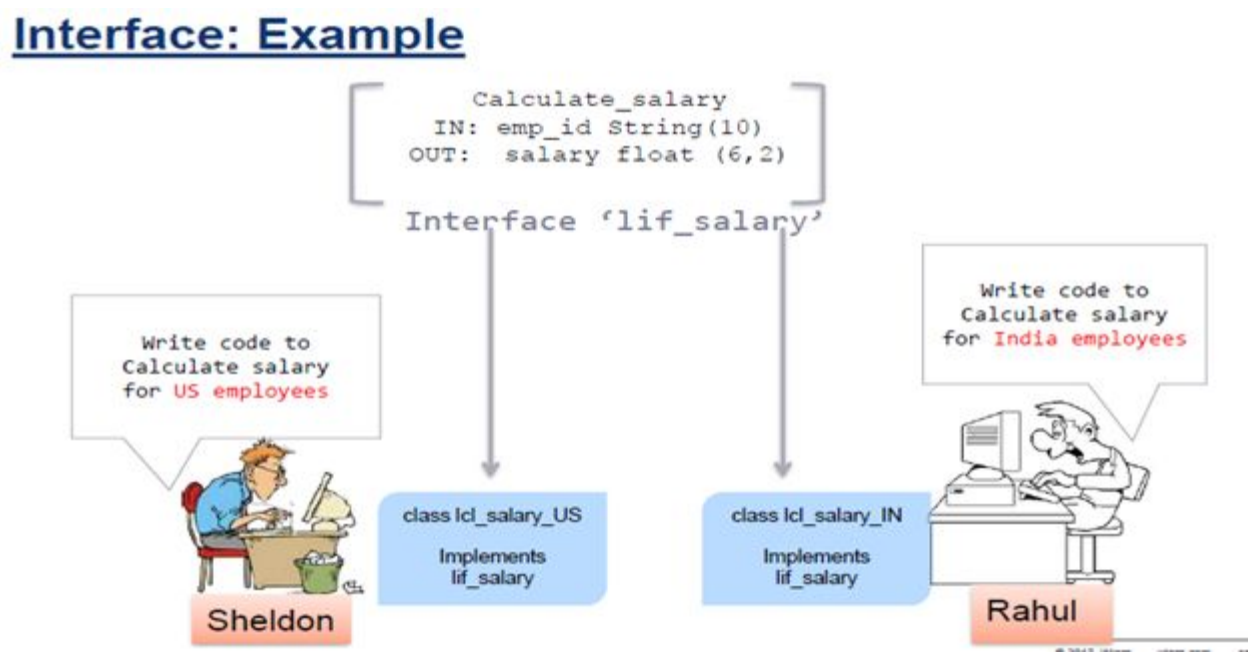


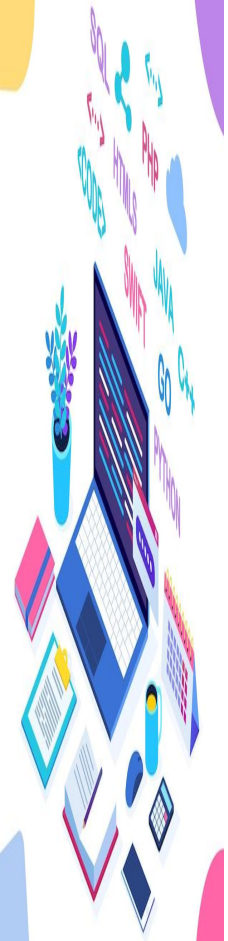
Interfaces

- An interface is a named collection of method declarations (without implementations)
  - An interface can also include constant declarations
  - An interface is syntactically similar to an abstract class
  - An interface is a collection of abstract methods and final variables
  - A class implements an interface using the implements clause

- An interface defines a protocol of behavior
- An interface lays the specification of what a class is supposed to do
- How the behavior is implemented is the responsibility of each implementing class
- Any class that implements an interface adheres to the protocol defined by the interface, and in the process, implements the specification laid down by the interface.

Interfaces


Interface: Example
Calculate_salary
IN: emp_id String(10)
OUT: salary float (6,2)
Interface 'lif_salary'
Write code to Calculate salary for US employees
Write code to Calculate salary for India employees
class lcl_salary_US
Implements lif_salary
class lcl_salary_IN
Implements lif_salary
Sheldon
Rahul



Why interfaces are required ?

- Interfaces allow you to implement common behaviors in different classes that are not related to each other.
- Interfaces are used to describe behaviors that are not specific to any particular kind of object, but common to several kind of objects.
- Defining an interface has the advantage that an interface definition stands apart from any class or class hierarchy.
- This makes it possible for any number of independent classes to implement the interface.
- Thus, an interface is a means of specifying a consistent specification, the implementation of which can be different across many independent and unrelated classes to suit the respective needs of such classes.
- Interfaces reduce coupling between components in your software.
- Java does not support multiple inheritance.
- This is a constraint in class design, as a class cannot achieve the functionality of two or more classes at a time.
- Interfaces help us make up for this loss as a class can implement more than one interface at a time.
- Thus, interfaces enable you to create richer classes and at the same time the classes need not be related.



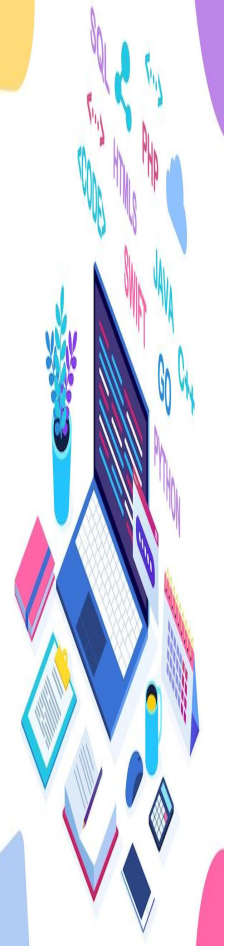
Interface members

- All the methods that are declared within an interface are always, by default, public and abstract.
- Any variable declared within. an interface is always, by default, public static and final.





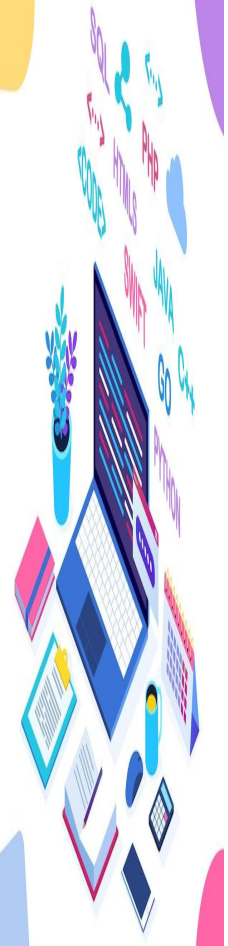
- What is the behavior which is common among the entities depicted in the pictures above?
  - ➢ **All of them can fly.**
- **Requirement :** You have to develop 3 classes, Bird, Superman and Aircraft with the condition that all these classes must have a method called **fly().**
- What is the mechanism, using which you can ensure that the method fly() is implemented in all these classes? **An Abstract class or An Interface?**



Defining an Interface

- An interface is syntactically similar to a class.
- It's general form is:

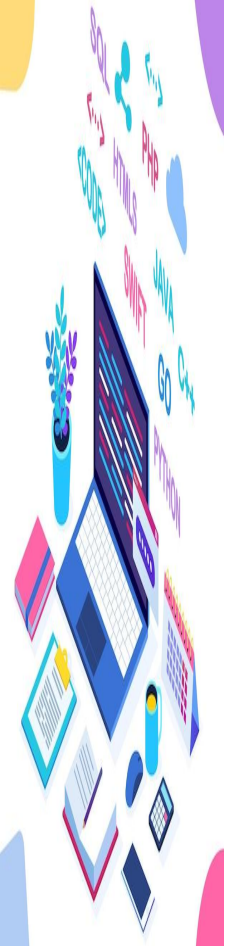
```
public interface FirstInterface {
int addMethod(int x, int y);
float divMethod(int m, int n);
void display();
int VAR1 = 10;
float VAR2 = 20.65;
}
```



Implementing Interfaces

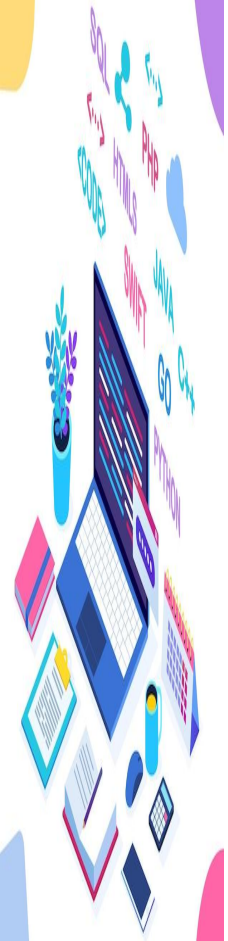
- A class implements an interface
- A class can implement more than one interface by giving a comma- separated list of Interfaces

```
class MyClass implements FirstInterface{
public int addMethod(int a, int b){
return(a+b);
}
public float divMethod(int i, int j){
return(i/j);
}
public void display(){
System.out.println("Variable 1 :" +VAR1);
System.out.println("Variable 2 :" +VAR2);
}
}
```



Applying Interfaces

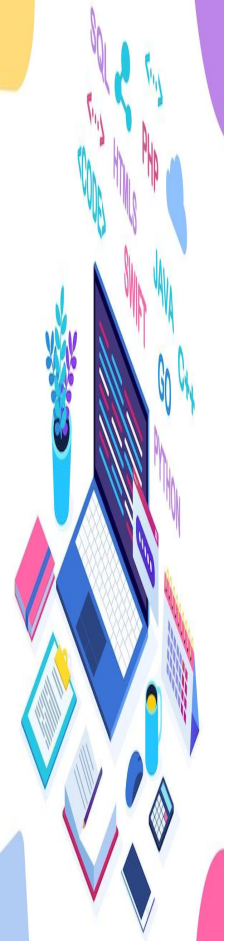
- Software development is a process where constant changes are likely to happen.
- There can be changes in requirement, changes in design, changes in implementation.
- Interfaces support change.
- Programming through interfaces helps create software solutions that are reusable, extensible, and maintainable.



Applying Interfaces

```
interface IntDemo{
void display();
}
class classOne implements IntDemo{
void add(int x, int y){
System.out.println("The sum is :" +(x+y));
}
public void display(){
System.out.println("Welcome to Interfaces");
}
}
class classTwo implements IntDemo{
void multiply(int i,int j, int k) {
System.out.println("The result:" +(i*j*k) );
}
public void display(){
System.out.println("Welcome to Java ");
}
}
```

```
class DemoClass{
public static void main(String args[]) {
classOne c1= new classOne();
c1.add(10,20);
c1.display();
classTwo c2 = new classTwo();
c2.multiply(5,10,15);
c2.display();
}
}
```



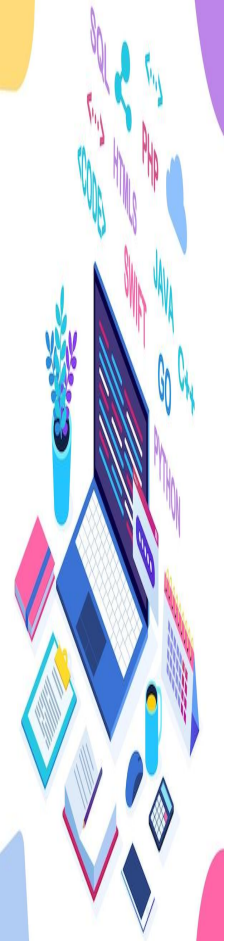
Interfaces

```
interface Vehicle {
    void start(); // abstract method
}
interface Engine {
    void fuelType();
}
class Car implements Vehicle, Engine {
    public void start() {
        System.out.println("Car started");
    }
    public void fuelType() {
        System.out.println("Petrol engine");
    }
}
```

```
public class Main {
    public static void main(String[] args)
{
        Car c = new Car();
        c.start();
        c.fuelType();
        Vehicle v;
        v=new Car();
        v.start();
        //v.fuelType(); -->error
        Engine e;
        e=new Car();
        e.fuelType();

    }
}
```

Nested Interfaces - within a class



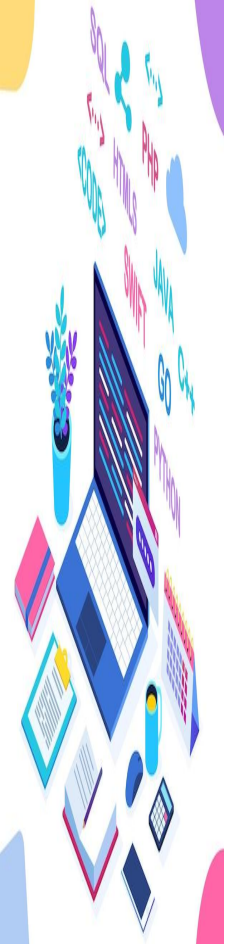
```
class Vehicle {
    interface Engine {
        void start();
    }
}

class Car implements Vehicle.Engine {
    public void start() {
        System.out.println("Car engine started");
    }
}

p
```

```
public class Main {
    public static void main(String[] args) {
        Vehicle.Engine engine = new Car();
        engine.start();
        Car c=new Car();
        c.start();

    }
}
```



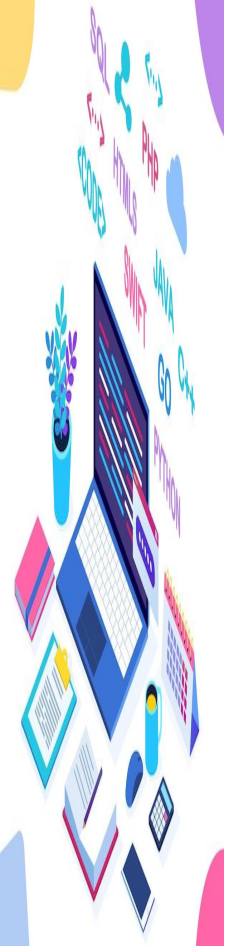
Nested Interfaces - within interface

```
interface Vehicle {
    void stop();
    interface Engine {
        void start();
    }
}

class Car implements Vehicle.Engine {
    public void start() {
        System.out.println("Car engine started");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Vehicle.Engine engine = new Car();
        engine.start();
        Car c=new Car();
        c.start();

    }
}
```

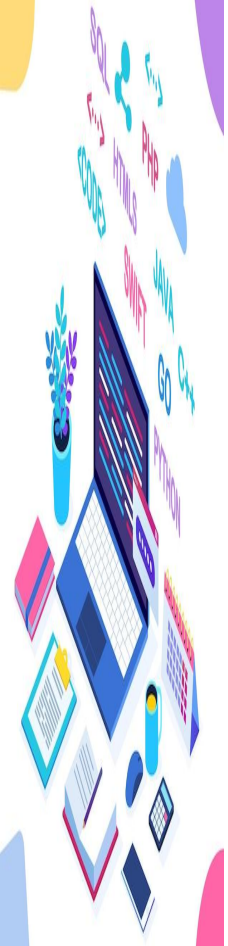


Nested Interfaces

```
interface Vehicle {
    void stop();
    interface Engine {
        void start();
    }
}
class Car implements Vehicle.Engine, Vehicle {
    public void start() {
        System.out.println("Car engine started");
    }
    public void stop()
    {
        System.out.println("Car engine stopped");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Vehicle.Engine engine = new Car();
        engine.start();
        Car c=new Car();
        c.start();
        c.stop();
        Vehicle v=new Car();
        v.stop();

    }
}
```

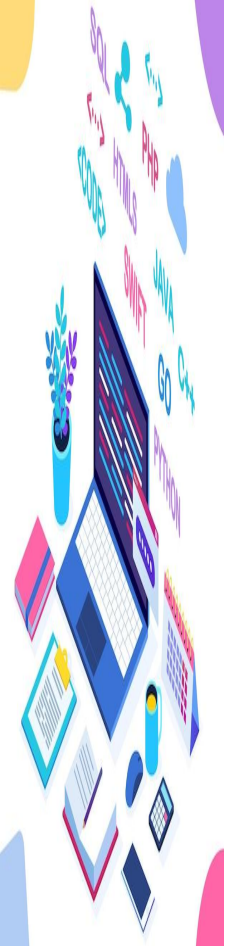


Hybrid Inheritance

```
interface A {
void methodA();
}
interface B extends A {
void methodB();
}

class C implements B {
    public void methodA() {
System.out.println("From A"); }
    public void methodB() {
System.out.println("From B"); }
}
```

```
public class Main {
    public static void main(String[] args) {
        C obj = new C();
        obj.methodA();
        obj.methodB();
    }
}
```

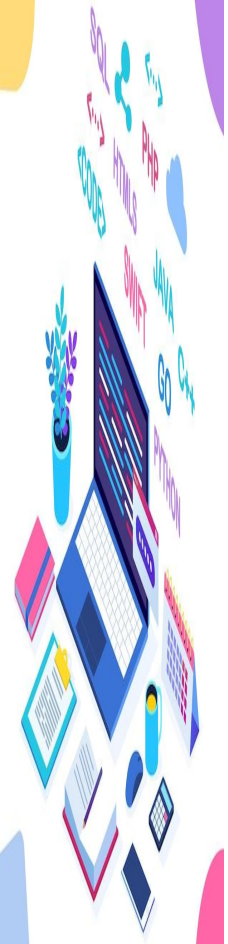


Interface

```
interface Calculator {
  static int add(int a, int b) {
     return a + b;
    }
default void multiply(int a, int b) {
    System.out.println("Multiplication: " + (a * b));
}
 void subtract(int a, int b);  // abstract method
}
class MyCalculator implements Calculator {
    public void subtract(int a, int b) {
        System.out.println("Subtraction: " + (a - b));
    }
}
```

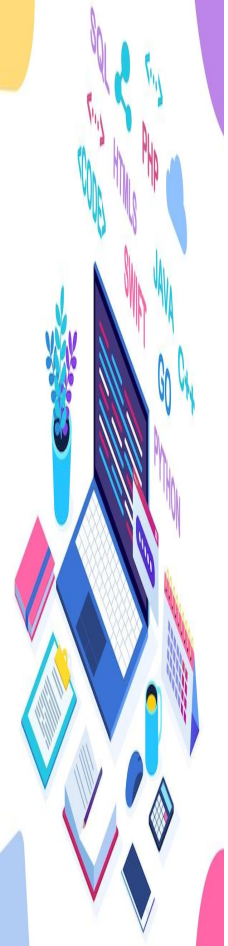
```
public class Main {
    public static void main(String[] args) {
        // Call static method directly from interface
        int sum = Calculator.add(10, 20);
        System.out.println("Sum: " + sum);

 // Create object to call default and abstract
methods
        MyCalculator calc = new MyCalculator();
        calc.multiply(5, 4);   // default method
        calc.subtract(15, 7);  // abstract method
    }
}
```



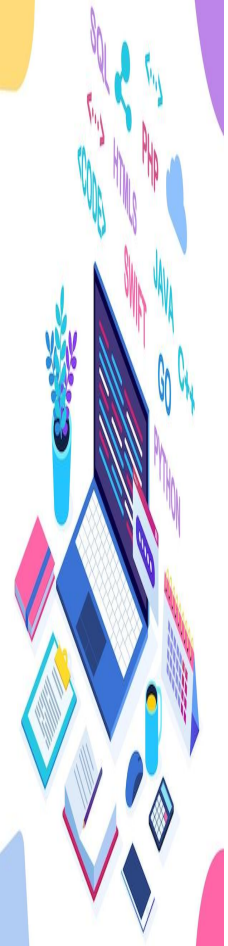
Exception Handling

- Similarly, when we write programs as part of an application, we may have to visualize the challenges that can disrupt the normal flow of execution of the code.
- Once we know what are the different situations that can disrupt the flow of execution, we can take preventive measures to overcome these disruptions.
- In java, this mechanism comes in the form of Exception Handling.
- An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions
- The ability of a program to intercept run-time errors, take corrective measures and continue execution is referred to as exception handling



Exception Handling

- There are various situations when an exception could occur:
  - Attempting to access a file that does not exist
  - Inserting an element into an array at a position that is not in its bounds
  - Performing some mathematical operation that is not permitted
  - Declaring an array using negative values

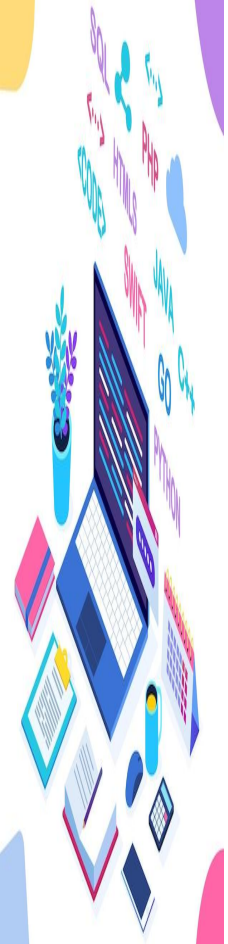


Uncaught Exceptions

```
class Demo {
public static void main(String args[]) {
int x = 0;
int y = 50/x;
System.out.println("y = " +y);
}
}
```

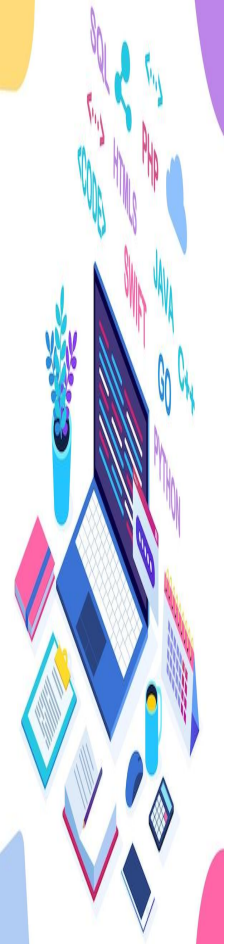
Although this program will compile, but when you execute it, the Java run-time-system will generate an exception and displays the following output on the console :

```
java.lang.ArithmeticException: / by zero
at Demo.main(Demo.java:4)
```



Exception Handling Techniques

- There are several built-in exception classes that are used to handle the very fundamental errors that may occur in your programs
- You can create your own exceptions also by extending the Exception class
- These are called user-defined exceptions, and will be used in situations that are unique to your applications



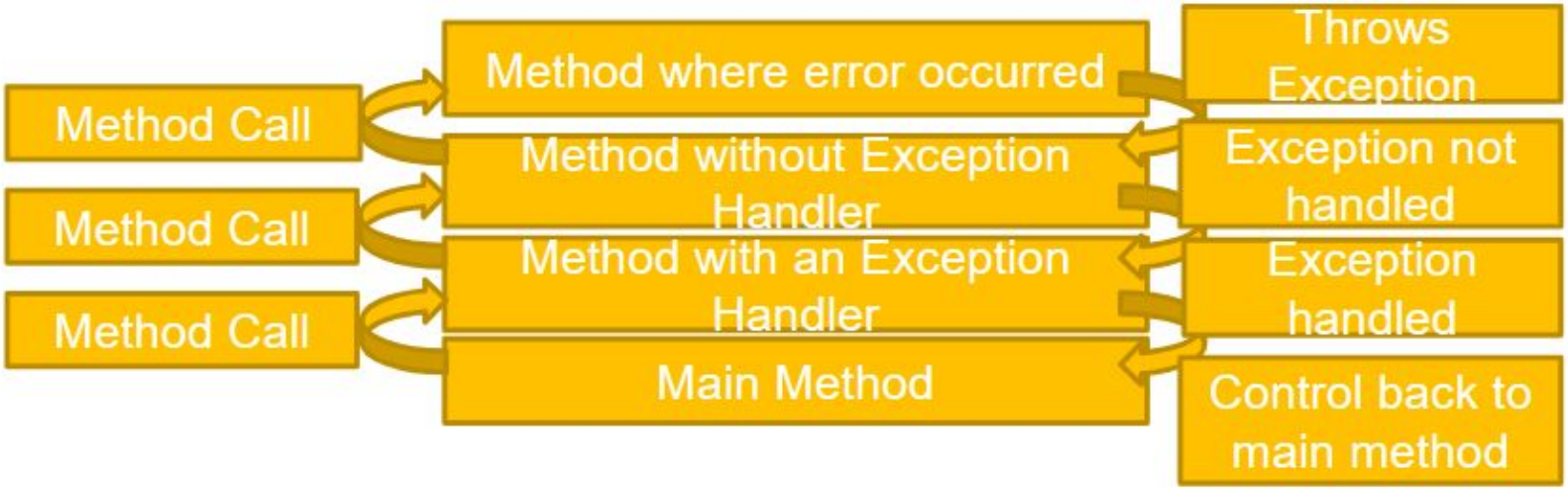
Handling Runtime Exceptions

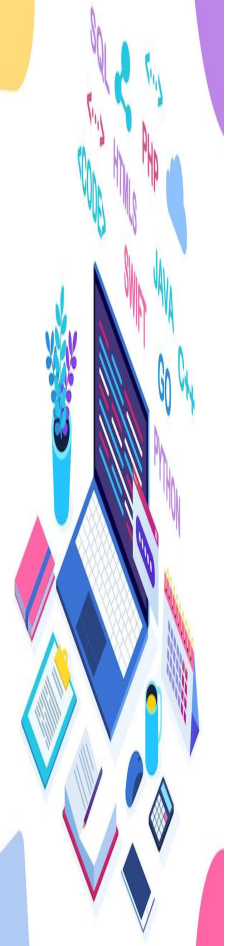
- Whenever an exception occurs in a program, an object representing that exception is created and thrown in the method in which the exception occurred
- Either you can handle the exception, or ignore it In the latter case, the exception is handled by the Java run-time-system and the program terminates
- However, handling the exceptions will allow you to fix it, and prevent the program from terminating abnormally



Advantages - Exceptions

1. Separating Error-Handling Code from "Regular" Code
2. Propagating Errors Up the Call Stack
3. Grouping and Differentiating Error Types

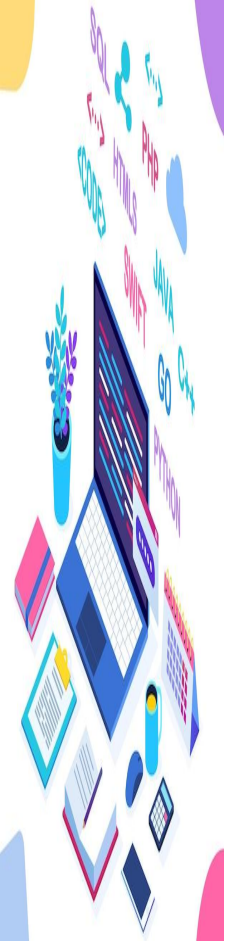




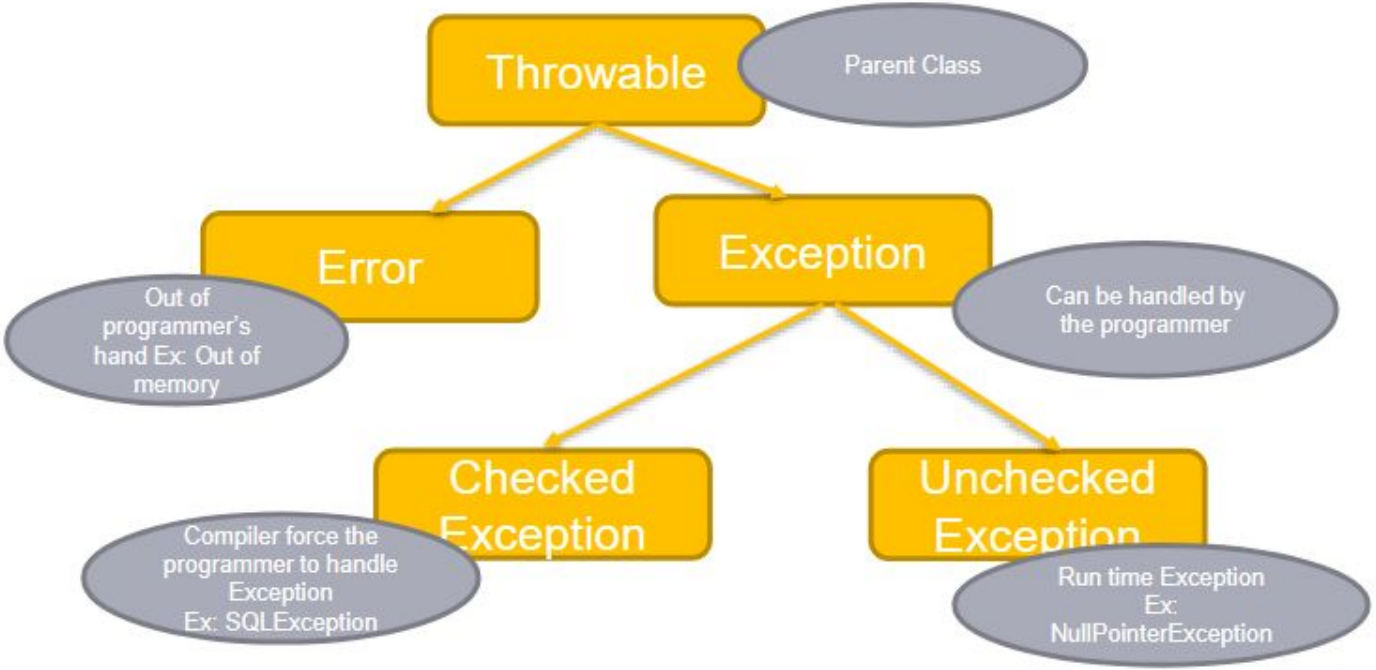
Exception Handling Keywords

Java's exception handling is managed using the following keywords: try, catch, throw, throws and finally.

```
try {
// code comes here
}
catch(TypeofException obj) {
//handle the exception
}
finally {
//code to be executed before the program ends
}
```



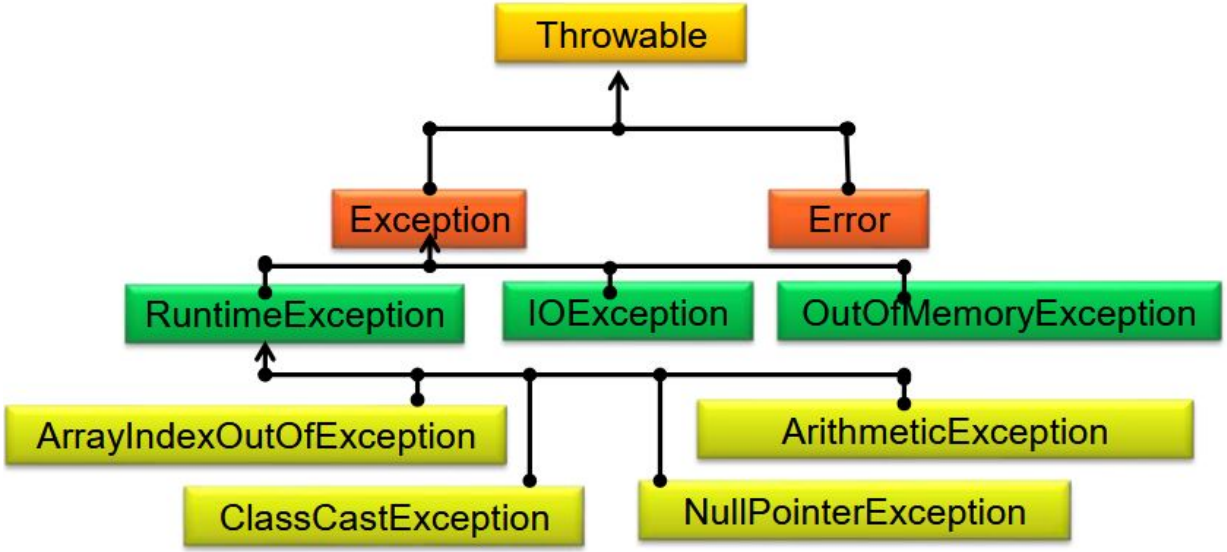
Exception Handling Keywords

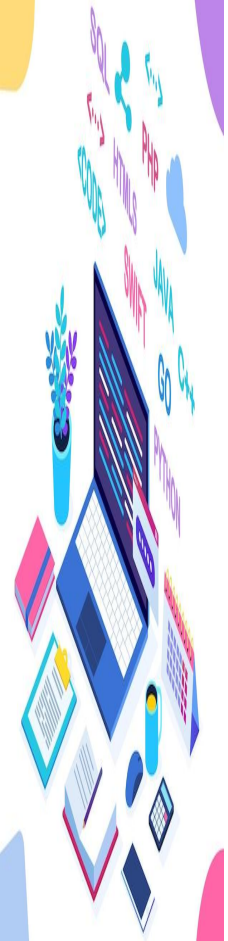




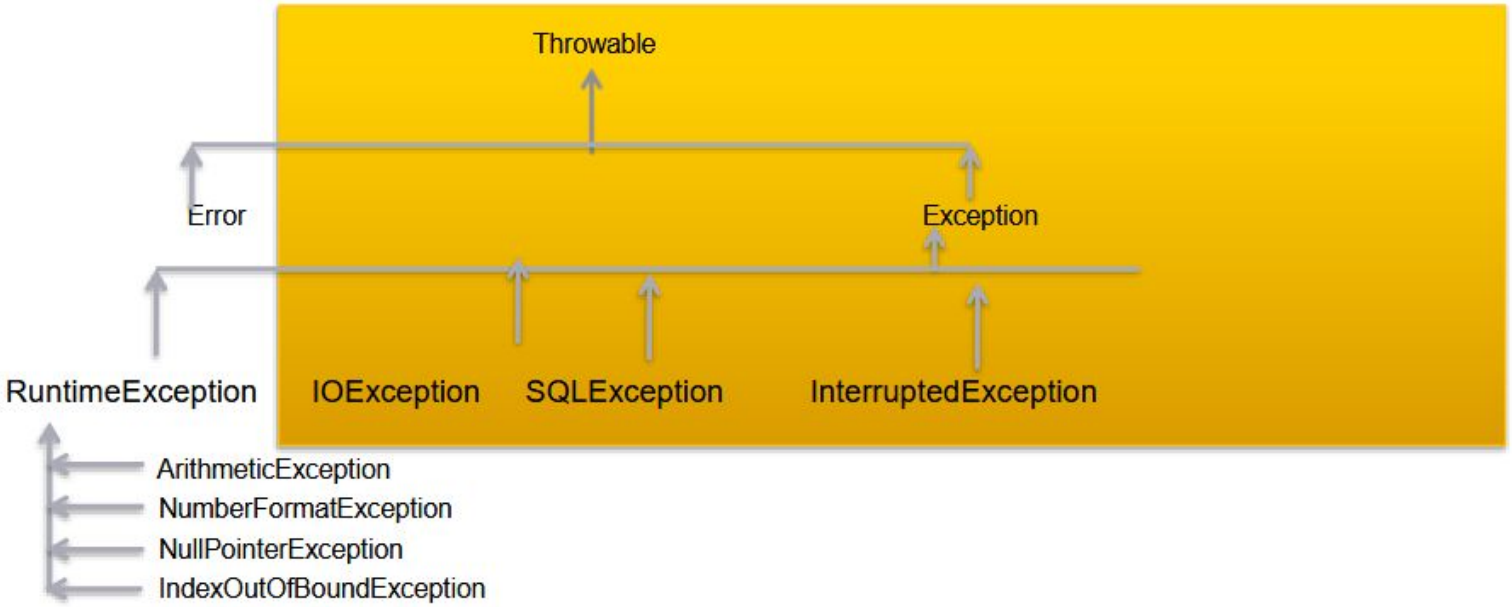
Exception Types

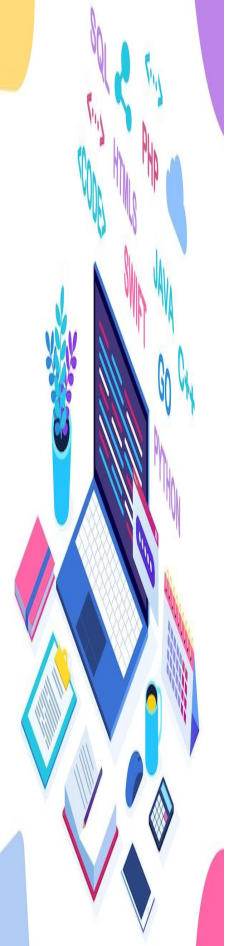
Exceptions are implemented in Java through a number of classes. The exception hierarchy is as follows:





Checked Exceptions





Checked Exceptions

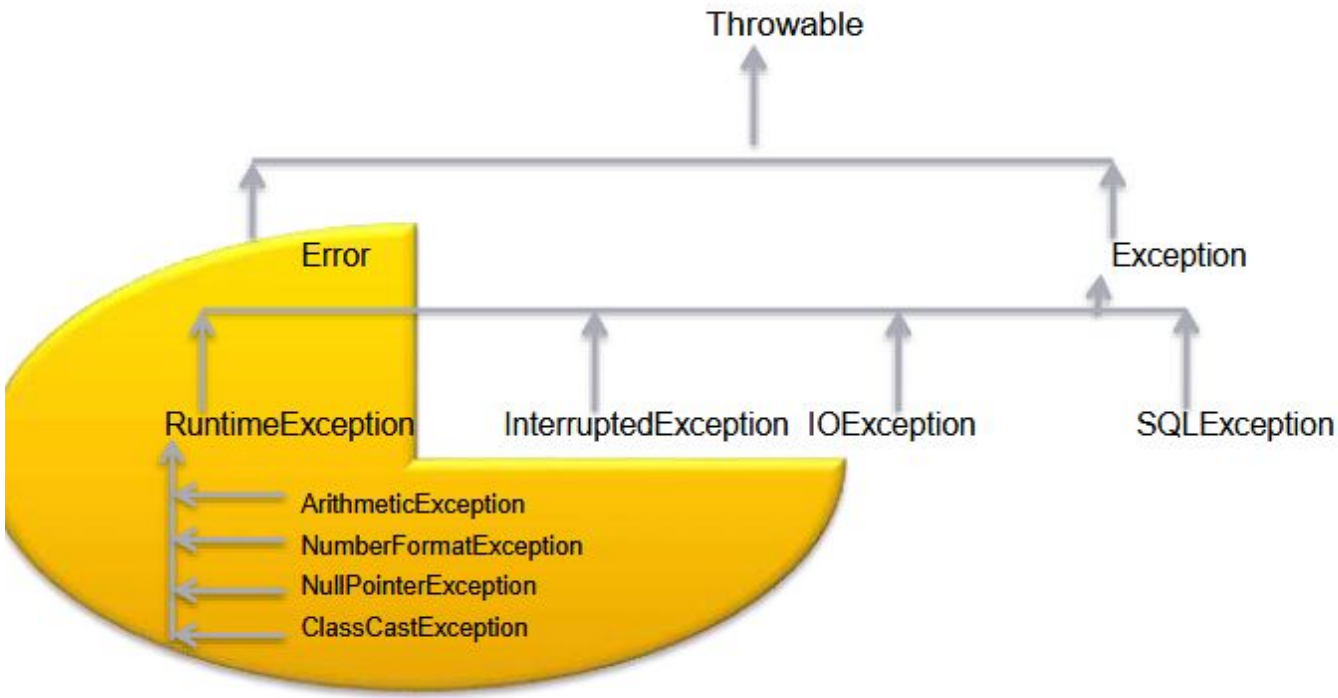
- A checked exception is an exception that usually happens due to user error or it is an error situation that cannot be foreseen by the programmer
- A checked exception must be handled using a try or catch or at least declared to be thrown using throws clause. Non compliance of this rule results in a compilation error

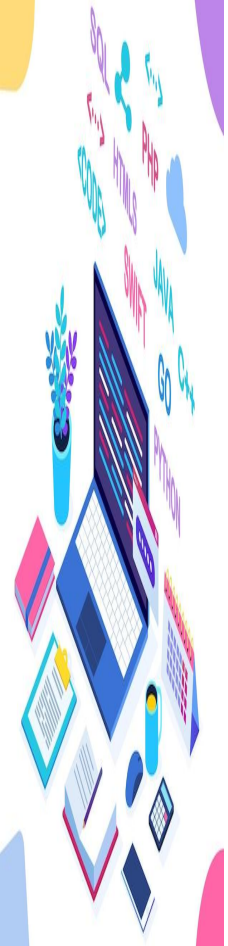
  Ex: FileNotFoundException
- If you try to open a file using

  FileInputStream fx = new FileInputStream("A1.txt");
- During execution, the system will throw a FileNotFoundException, if the file A1.txt is not located, which may be beyond the control of a programmer



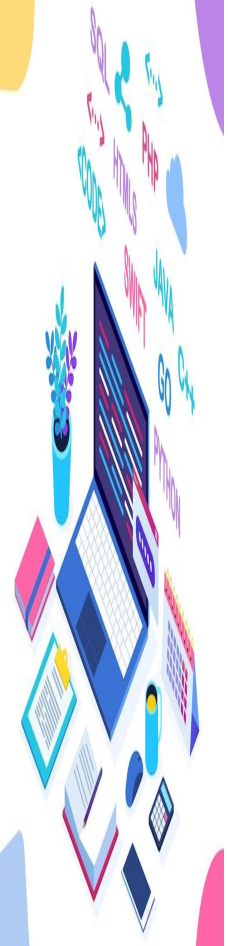
Unchecked Exceptions





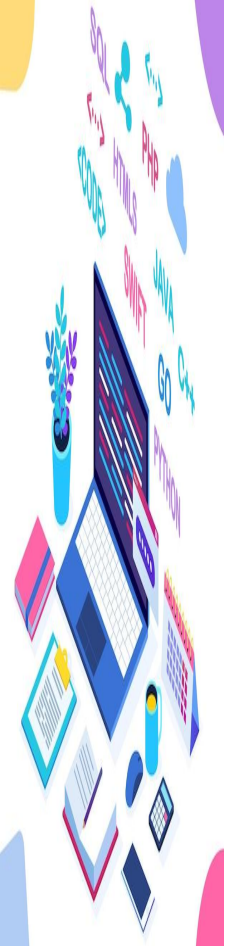
Unchecked Exceptions

- An unchecked exception is an exception, which could have been avoided by the programmer
- The class RuntimeException and all its subclasses are categorized as Unchecked Exceptions
- If there is any chance of an unchecked exception occuring in the code, it is ignored during compilation



Error

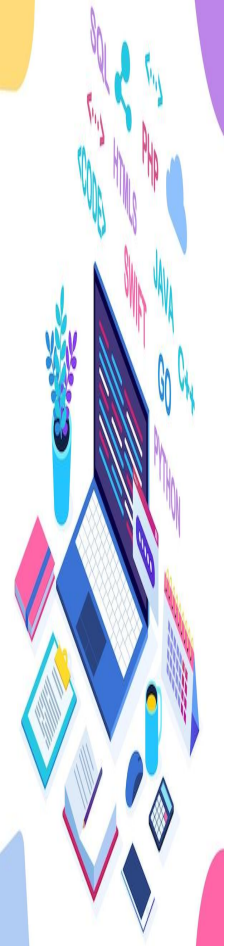
- Error is not considered as an Exception
- Errors are problems that arise beyond the control of the programmer or the user
- A programmer can rarely do anything about an Error that occurs during the execution of a program
- This is the precise reason Errors are typically ignored in the code
- Errors are also ignored by the compiler
- Ex : Stack Overflow



Example – StackOverflowError

```
public class Tester {
public static void recursivePrint(int num) {
System.out.println("Number: " + num);
if(num == 0)
return;
else
recursivePrint(++num);
}
public static void main(String[] args) {
Tester.recursivePrint(1);
}
}
```

Exception in thread "main" java.lang.StackOverflowError

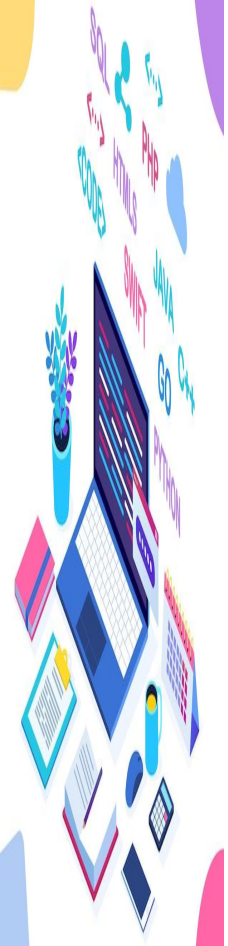


Activity

Listed below are some of the built in exception classes. List them in the table below as per their classification, whether they are checked exceptions or unchecked exceptions :

1)NullPointerException 2) ClassNotFoundException 3) IOException
4) InterruptedException 5)ArrayIndexOutOfBoundsException
6) NumberFormatException 7) ClassCastException 8)SQLException
9) IllegalAccessException 10) NegativeArraySizeException

Checked Exception	Unchecked Exception



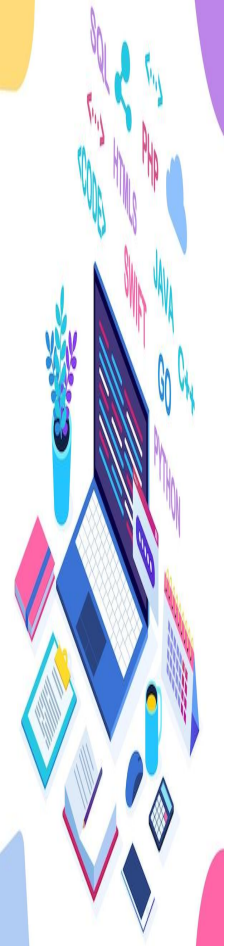
Try-Catch Block

- Any part of the code that can generate an error should be put in the try block
- Any error should be handled in the catch block defined by the catch clause
- This block is also called the catch block, or the exception handler
- The corrective action to handle the exception should be put in the catch block



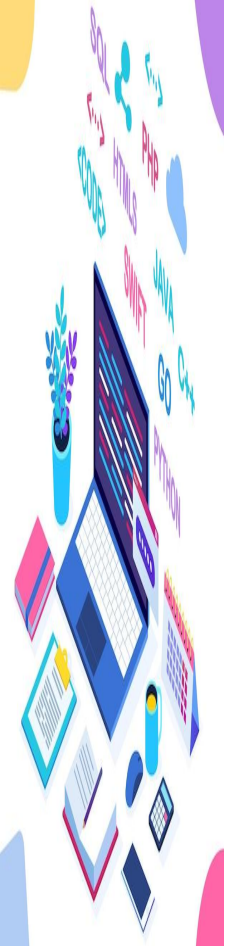
How to Handle exceptions?

```
class ExceptDemo{
public static void main(String args[]){
int x, a;
try{
x = 0;
a = 22 / x;
System.out.println("This will be bypassed.");
}
catch (ArithmeticException e){
System.out.println("Division by zero.");
}
System.out.println("After catch statement.");
```



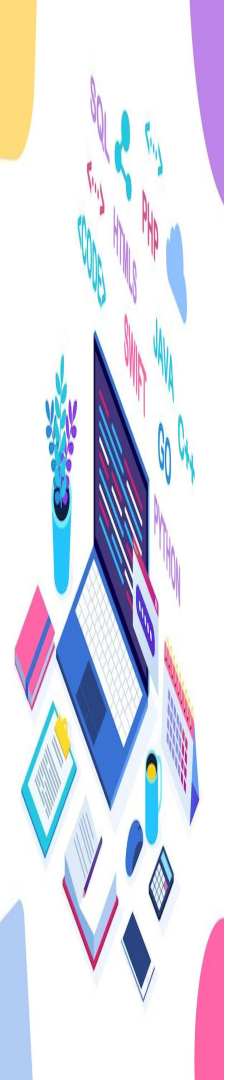
Multiple Catch Statements

- A single block of code can raise more than one exception
- You can specify two or more catch clauses, each catching a different type of execution
- When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed
- After one catch statement executes, the others are bypassed, and execution continues after the try/catch block



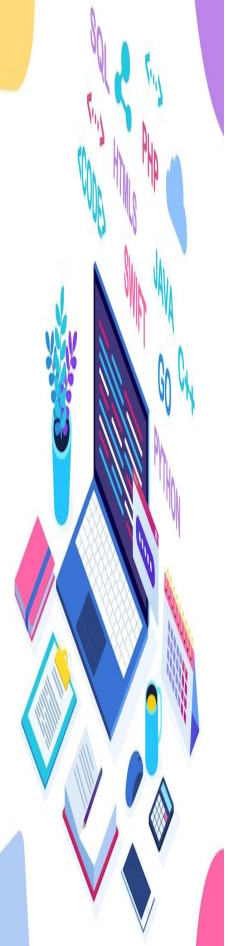
Multiple Catch Statements

```
class MultiCatch{
public static void main(String args[]){
try{
int l = args.length;
System.out.println("l = " +l);
int b = 42 / l;
int arr[] = { 1 };
arr[22] = 99;
}
catch(ArithmeticException e){
System.out.println("Divide by 0: "+ e);
}
catch(ArrayIndexOutOfBoundsExcepti
on e){
System.out.println("Array index oob:
"+e);
}
System.out.println("After try/catch
blocks.");
}
}
```



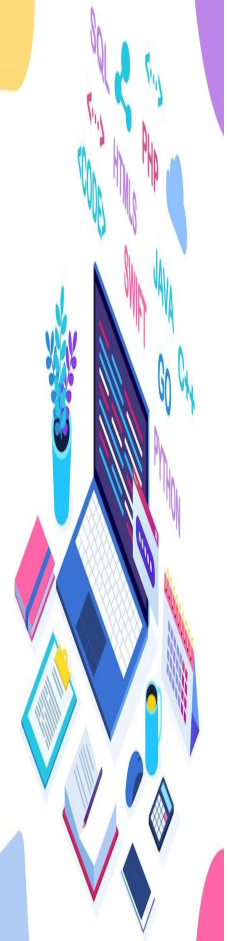
Multiple Catch Statements involving Exception Superclasses & Subclasses

- When you use multiple catch statements, it is important to remember that exception subclasses must come before any of their exception superclasses
- This is because a catch statement that uses a superclass will catch exceptions of that type as well as exceptions of its subclasses
- Thus, a subclass exception would never be reached if it came after its superclass that manifests as an unreachable code error



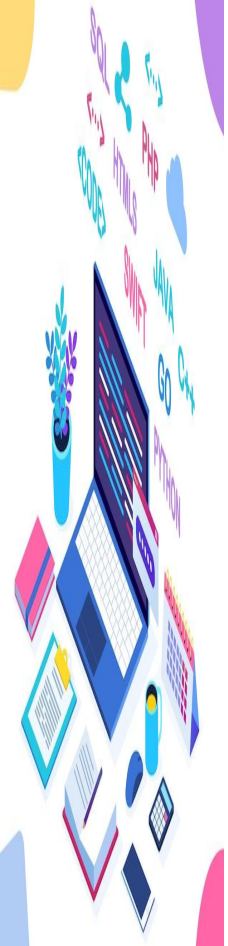
Nested try Statements

- The try statement can be nested
- If an inner try statement does not have a catch handler for a particular exception, the outer block's catch handler will handle the exception
- This continues until one of the catch statement succeeds, or until all of the nested try statements are exhausted
- If no catch statement matches, then the Java runtime system will handle the exception



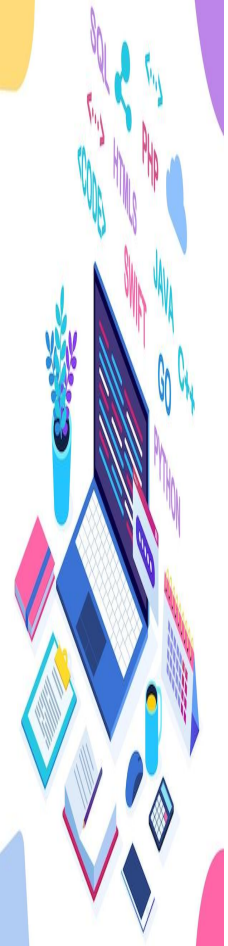
Nested try Statements

```
try
{
statement 1;
statement 2;
try
{
statement 1;
statement 2;
}
catch(Exception e)
{
```



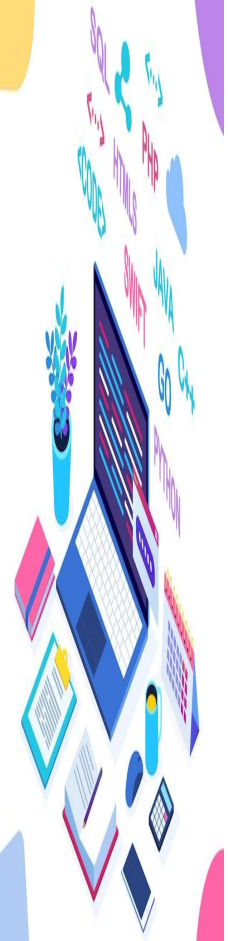
Nested try Statements

```
class Nested_Try{
public static void main(String args[]){
try{
try{
System.out.println(“Arithmetic Division");
int b =39/0;
} catch(ArithmeticException e){
System.out.println(e);
}
try{
int a[]=new int[5];
System.out.println(“Accessing Array Elements");
```



Using throw

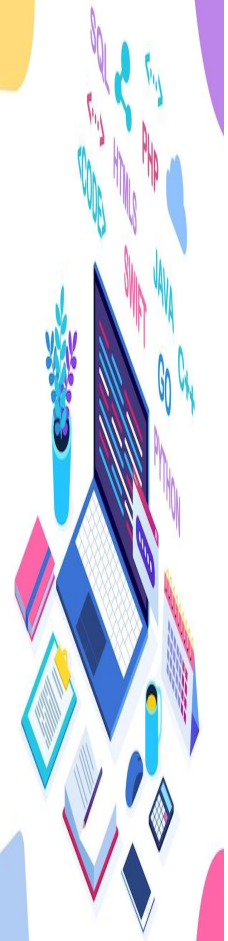
- Sometimes, a method is capable of causing an exception that it does not handle
- Then, it must specify this behavior so that callers of the method can guard themselves against that exception
- While declaring such methods, you have to specify what type of exception it may throw by using the throws keyword
- A throws clause specifies a comma-separated list of exception types that a method might throw:
- type method-name(parameter list) throws exception-list



Using throw

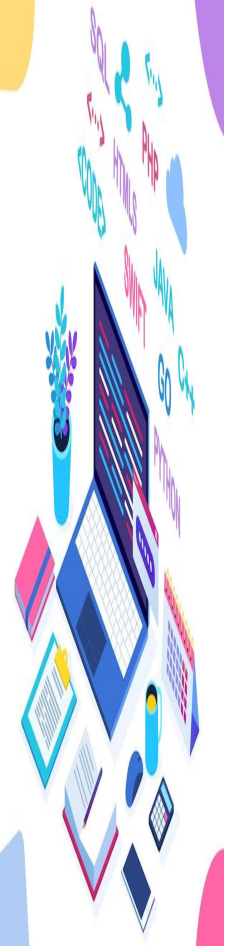
```
class ThrowsDemo{
static void throwOne(){
System.out.println("Inside throwOne.");
throw new FileNotFoundException();
}
public static void main(String args[]){
throwOne();
}
}
```

What happens when this code is compiled ?
Compilation Error……why?



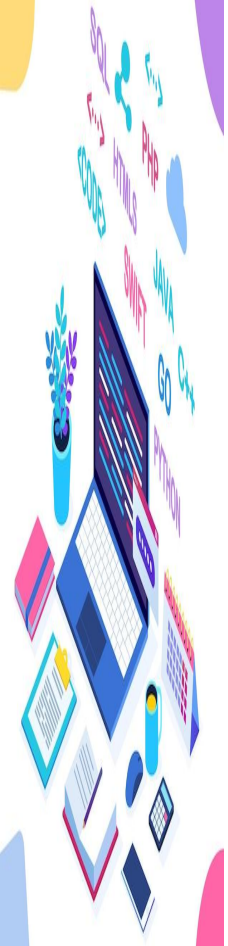
Implementing throws

```
import java.io.*;
class ThrowsDemo{
static void throwOne() throws FileNotFoundException{
System.out.println("Inside throwOne.");
throw new FileNotFoundException();
}
public static void main(String args[]) {
try{
throwOne();
}
catch (FileNotFoundException e){
```



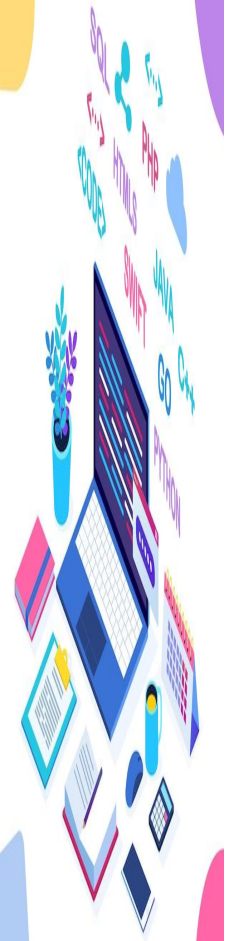
Rule governing overriding method with throws

- The overriding method must NOT throw checked exceptions that are new or broader than those declared by the overridden method
- For eg : A method that declares(throws) an SQLException cannot be overriden by a method that declares an IOException, Exception or any other exception unless it is a subclass of SQLException
- In other words, if a method declares to throw a given exception, the overriding method in a subclass can only declare to throw the same exception or its subclass
- This rule does not apply for unchecked exceptions



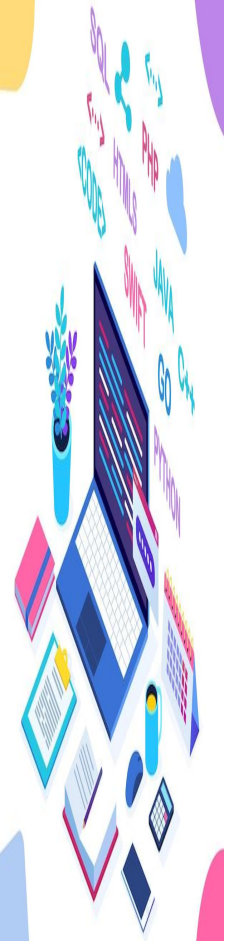
Using throw

- System-generated exceptions are thrown automatically
- At times you may want to throw the exceptions explicitly which can be done using the throw keyword
- The exception-handler is also in the same block
- The general form of throw is:
- throw ThrowableInstance
- Here, ThrowableInstance must be an object of type Throwable, or a subclass of Throwable



Using throw

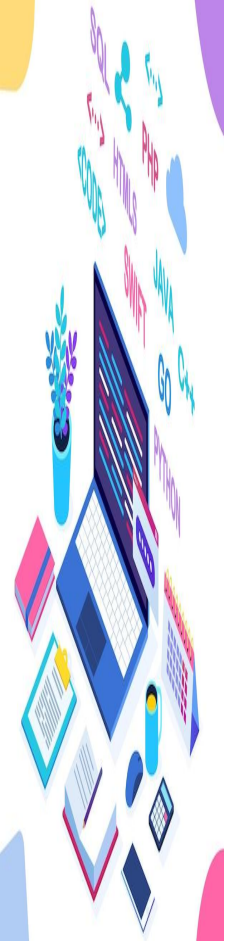
```
class ThrowDemo{
public static void main(String args[]) {
try {
int age=Integer.parseInt(args[0]);
if(age < 18)
throw new ArithmeticException();
else
if(age >=60)
throw new ArithmeticException("Employee is retired");
}
catch(ArithmeticException e) {
```



User Defined Exceptions

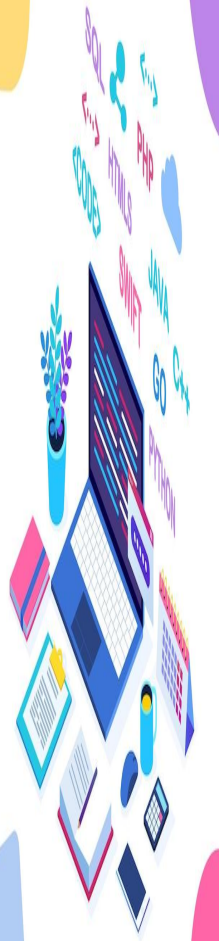
- Java provides extensive set of in-built exceptions
- But there may be cases where we may have to define our own exceptions which are application specific
- For ex: If we have are creating an application for handling the database of eligible voters, the age should be greater than or equal to 18
- In this case, we can create a user defined exception, which will be thrown in case the age entered is less than 18
- While creating user defined exceptions, the following aspects have to be taken care :
- The user defined exception class should extend from the Exception class and its subclass

User Defined Exceptions

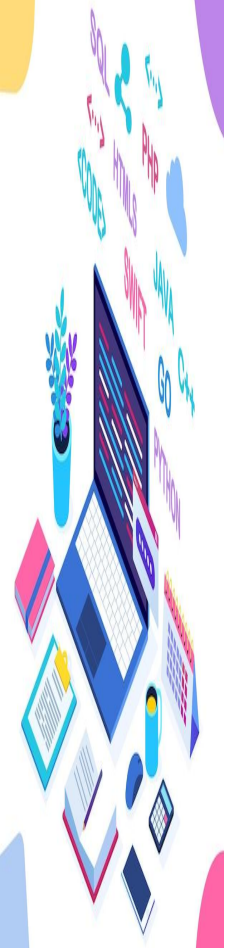


```
class MyExceptionDemo{
static int flag=0;
public static void main(String args[]) {
try {
int age=Integer.parseInt(args[0]);
if(age < 18)
throw new MyException();
}
catch(ArrayIndexOutOfBoundsException e) {
flag=1;
System.out.println("Exception : "+ e);
```

User Defined Exceptions

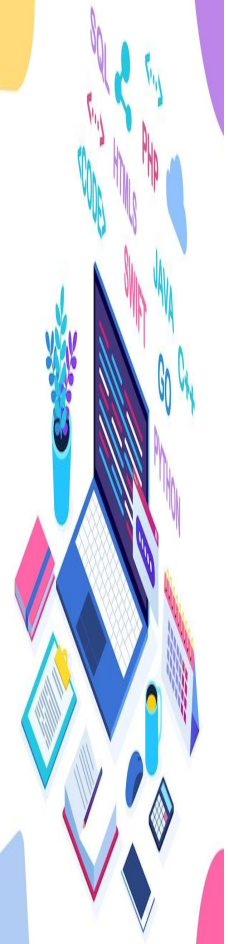


```
catch(NumberFormatException e) {
flag=1;
System.out.println("Exception : "+ e);
}
catch (MyExceptionClass e) {
flag=1;
System.out.println("Exception : "+ e);
}
if(flag==0)
System.out.println("Everything is fine");
}
```



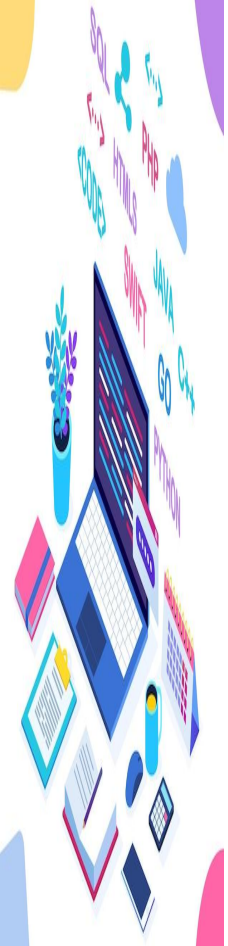
User Defined Exceptions

```
class MyException extends Exception {
public MyException() {
System.out.println("User defined Exception thrown");
}
public String toString() {
return "MyException Object : Age cannot be < 18 " ;
}
}
```



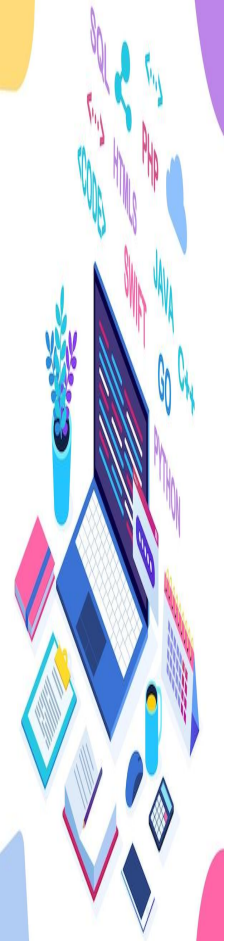
Using finally

- When an exception occurs, the execution of the program takes a non-linear path, and could bypass certain statements
- A program establishes a connection with a database, and an exception occurs
- The program terminates, but the connection is still open
- To close the connection, finally block should be used
- The finally block is guaranteed to execute in all circumstances



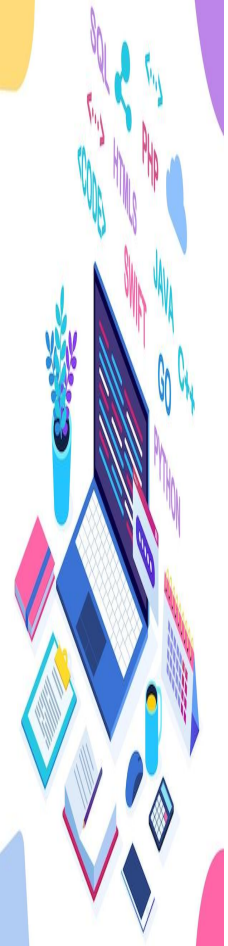
Using finally

```
import java.io.*;
class FinallyDemo{
static void funcA() throws FileNotFoundException
{
try{
System.out.println("inside funcA( )");
throw new FileNotFoundException( );
}
finally{
System.out.println
("inside finally of funA( )");
```



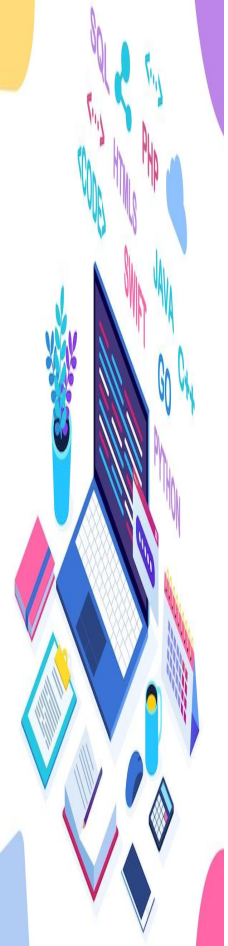
Using finally

```
static void funcB(){
try{
System.out.println("inside funcB( )");
}
finally{
System.out.println
("inside finally of funB( )");
}
}
}
```



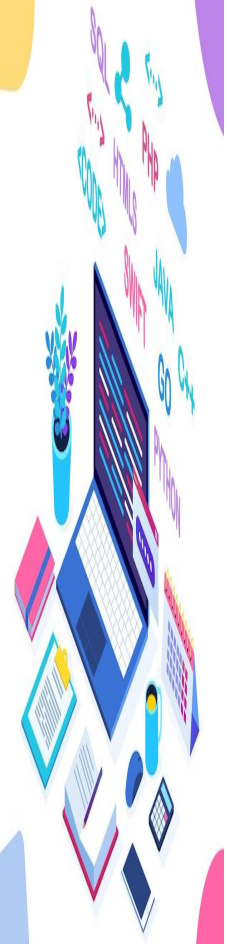
Using finally

```
static void funcC() {
try{
System.out.println("inside funcC( )");
}
finally{
System.out.println
("inside finally of funcC( )");
}
}
```



Using finally

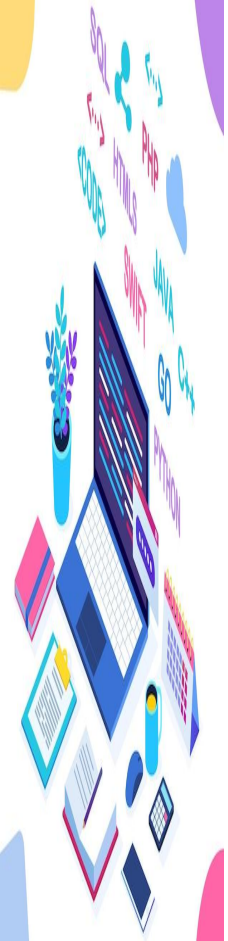
```
public static void main(String args[]){
try{
funcA();
}
catch (Exception e){
System.out.println("Exception caught");
}
funcB( );
funcC( );
}
}
```



Significance of printStackTrace() method

We can use the printStackTrace() method to print the program's execution stack

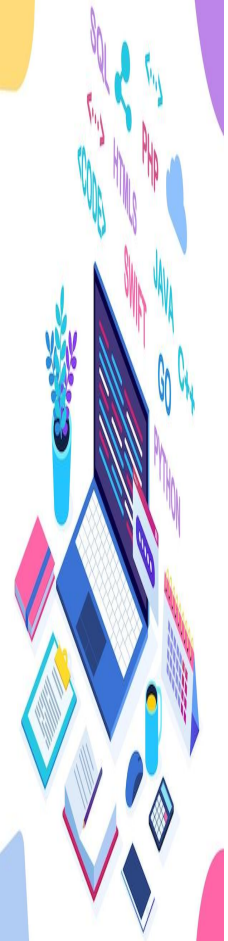
This method is used for debugging



Significance of printStackTrace() method

```
import java.io.*;
class PrintStackExample {
public static void main(String args[]) {
try {
m1();
}
catch(IOException e) {
e.printStackTrace();
}
}
```

Significance of printStackTrace() method



```
static void m1() throws IOException {
m2();
}
static void m2() throws IOException {
m3();
}
static void m3() throws IOException{
throw new IOException();
}
}
```

Expected Output

```
java.io.IOException
at PrintStackExample.m3(PrintStackExample.java:24)
at PrintStackExample.m2(PrintStackExample.java:20)
at PrintStackExample.m1(PrintStackExample.java:16)
at PrintStackExample.main(PrintStackExample.java:5)
```